MongoDB CRUD Commands – Definition, Syntax & Example

1. use <database_name>

Definition
Creates a database (if it doesn't exist) or switches to an existing database.

Syntax
use database_name

Example
use DB_One

```
DB_Two> use DB_One
switched to db DB_One
DB_One>
```

// Switched from database – DB_Two to exisiting database – DB_One

2. db

Definition
Displays the currently selected database.

Syntax
db

Example
db

```
DB_One> db
DB_One
DB_One>
```

3. db.createCollection()

Definition
Creates a new collection.

Syntax
db.createCollection("collection_name")

Example
db.createCollection("New")

```
DB_One> db.createCollection("New")
{ ok: 1 }
DB_One>
```

4. show collections

Definition
Displays all collections in the current database.

Syntax
show collections

Example
show collections

```
DB_One> show collections
First
New
DB_One>
```

5. insertOne()

Definition
Inserts a single document into a collection.

Syntax
db.collection_name.insertOne({
  field1:value1,
  field2:value2
})

Example
db.New.insertOne({
  name:"Vihaana",
  dept:"IT",
  age:20
})

```
DB_One> db.New.insertOne({
|     name:"Vihaana",
|     dept:"IT",
|     age:20
| })
|
{
  acknowledged: true,
  insertedId: ObjectId('6a1fc4baa3b58f3212abc119')
}
DB_One>
```

6. insertMany()

Definition
Inserts multiple documents at once.

Syntax
```
db.collection_name.insertMany([
  {document1},
  {document2}
])
```

Example
```
db.New.insertMany([
  {
    name:"Yashvi",
    dept:"IT",
    age : 20
  },
  {
    name:"Anvika",
    dept:"IT",
    age : 20
  },
  {
    name:"Latchaya",
    dept:"IT",
    age : 20
  }
])
```

```
DB_One> db.New.insertMany([
|    {
|       name:"Yashvi",
|       dept:"IT",
|       age : 20
|    },
|    {
|       name:"Anvika",
|       dept:"IT",
|       age : 20
|    },
|    {
|       name:"Latchaya",
|       dept:"IT",
|       age : 20
|    }
| ])
|
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a1fc545a3b58f3212abc11a'),
    '1': ObjectId('6a1fc545a3b58f3212abc11b'),
    '2': ObjectId('6a1fc545a3b58f3212abc11c')
  }
}
DB_One>
```

8. find()

Definition
Retrieves all matching documents from a collection.

Syntax
db.collection_name.find()

Example
db.New.find()

```
DB_One> db.New.find()
[
  {
    _id: ObjectId('6a1fc4baa3b58f3212abc119'),
    name: 'Vihaana',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11a'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11b'),
    name: 'Anvika',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11c'),
    name: 'Latchaya',
    dept: 'IT',
    age: 20
  }
]
DB_One>
```

9. findOne()

Definition
Returns the first matching document.

Syntax
db.collection_name.findOne()

Example
db.New.findOne()

```
DB_One> db.New.findOne()
{
  _id: ObjectId('6a1fc4baa3b58f3212abc119'),
  name: 'Vihaana',
  dept: 'IT',
  age: 20
}
```

find() with one / more conditions (AND)

Definition
Retrieve documents that satisfy all the specified condition
“ , “ → acts as a AND operator here

Syntax
db.New.find({conditions})

Example
db.New.find({name:"Vihaana",age:20})

```
DB_One> db.New.find({name:"Vihaana",age:20})
[
  {
    _id: ObjectId('6a1fc4baa3b58f3212abc119'),
    name: 'Vihaana',
    dept: 'IT',
    age: 20
  }
]
```

find() with one / more conditions (OR)

Definition
Retrieve documents that satisfy at least one of the specified condition

Syntax

```
db.collection_name.find({
  $or: [
    { condition1 },
    { condition2 }
  ]
})
```

Example

```
db.New.find({ $or: [ { name: "Vihaana" }, { age: 20 } ] })
```

```
DB_One> db.New.find({ $or: [ { name: "Vihaana" }, { age: 20 } ] })
[
  {
    _id: ObjectId('6a1fc4baa3b58f3212abc119'),
    name: 'Vihaana',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11a'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11b'),
    name: 'Anvika',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11c'),
    name: 'Latchaya',
    dept: 'IT',
    age: 20
  }
]
DB_One>
```

find() with gt

Definition

Returns documents where the field value is greater than the specified value.

Syntax

```
db.collection.find({
  field: { $gt: value }
})
```

Example
db.New.find({ age: { $gt:20}})

```
DB_One> db.New.find({ age: { $gt:20}})
[
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11e'),
    name: 'Arun',
    dept: 'HR',
    age: 30
  }
]
```

find() with gte

Definition
Returns documents where the field value is greater than or equal to the specified value.

Syntax
db.collection.find({
  field: { $gte: value }
})

Example
db.New.find({ age: { $gte:20}})

```
DB_One> db.New.find({ age: { $gte:20}})
[
  {
    _id: ObjectId('6a1fc4baa3b58f3212abc119'),
    name: 'Vihaana',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11a'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11b'),
    name: 'Anvika',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11c'),
    name: 'Latchaya',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11e'),
    name: 'Arun',
    dept: 'HR',
    age: 30
  }
]
DB_One>
```

find() with lt

Definition

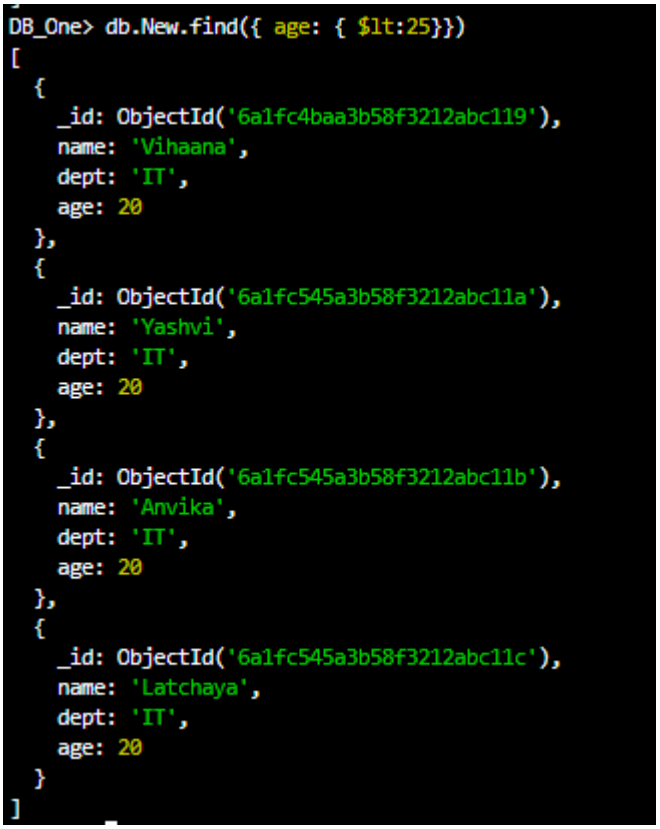
Returns documents where the field value is lesser than the specified value.

Syntax

db.collection.find({
  field: { $lt: value }
})

Example

db.New.find({ age: { $lt:25}})



find() with lte

Definition
Returns documents where the field value is less than or equal to the specified value.

Syntax
db.collection.find({
  field: { $lte: value }
})

Example
db.New.find({ age: { $lte:25}})

```
DB_One> db.New.find({ age: { $lte:25}})
[
  {
    _id: ObjectId('6a1fc4baa3b58f3212abc119'),
    name: 'Vihaana',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11a'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11b'),
    name: 'Anvika',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11c'),
    name: 'Latchaya',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  }
]
```

10. show dbs

Definition
Displays all databases present in MongoDB.

Syntax
show dbs

Example
show dbs

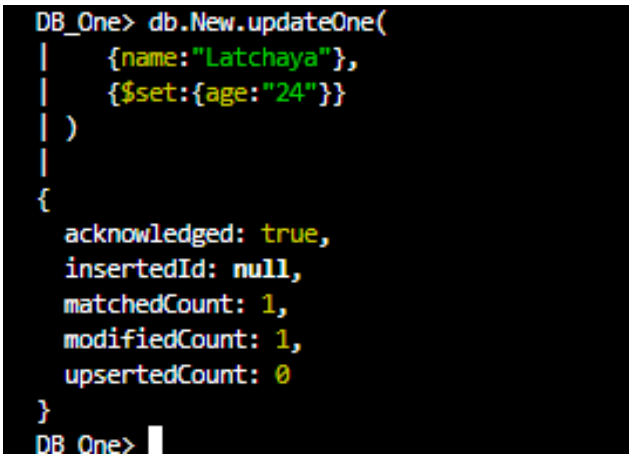
```
DB_One> show dbs
DB_One  128.00 KiB
DB_Two   40.00 KiB
admin    40.00 KiB
config  108.00 KiB
local    72.00 KiB
DB_One>
```

11. updateOne()

Definition
Updates one document that matches the condition.

Syntax
db.collection_name.updateOne(
  {condition},
  {$set:{field:value}}
)

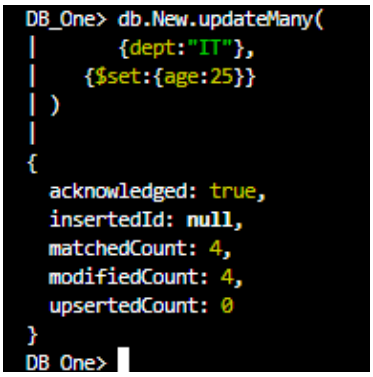
Example
db.New.updateOne(
  {name:"Latchaya"},
  {$set:{age:"24"}}
)



12. updateMany()

Definition
Updates multiple documents matching the condition.

Syntax
db.collection_name.updateMany(
  {condition},
  {$set:{field:value}}
)
Example
db.New.updateMany(
  {dept:"IT"},
  {$set:{age:25}}
)



13. replaceOne()

Definition
Replaces an entire document with a new document.

Syntax
db.collection_name.replaceOne(
{condition},
{new_document}
)

Example
db.students.replaceOne(
{name:"Latchaya"},
{
name:"Latchaya",
dept:"AIDS",
age:20
}
)

14. deleteOne()

Definition
Deletes one matching document.

Syntax
db.collection_name.deleteOne(
{condition}
)

Example
db.students.deleteOne({name:"Yashvi"})

```
DB_One> db.New.deleteOne({ name: "Yashvi" })
{ acknowledged: true, deletedCount: 1 }
DB_One>
```

```
DB_One> db.New.deleteOne({ name: "Yashvi" })
{ acknowledged: true, deletedCount: 1 }
DB_One> db.New.find()
[
  {
    _id: ObjectId('6a1fc4baa3b58f3212abc119'),
    name: 'Vihaana',
    dept: 'IT',
    age: 25
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11b'),
    name: 'Anvika',
    dept: 'IT',
    age: 25
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11c'),
    name: 'Latchaya',
    dept: 'IT',
    age: 25
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11e'),
    name: 'Arun',
    dept: 'HR',
    age: 30
  }
]
DB_One>
```

15. deleteMany()

Definition
Deletes multiple matching documents.

Syntax
db.collection_name.deleteMany(
  {condition}
)

Example
db.New.deleteMany({dept:"IT"})

```
DB_One> db.New.deleteMany( {dept:"IT"} )
{ acknowledged: true, deletedCount: 3 }
DB_One> db.New.find()
[
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11e'),
    name: 'Arun',
    dept: 'HR',
    age: 30
  }
]
DB_One>
```

16. countDocuments()

Definition
Counts the number of documents in a collection.

Syntax
db.collection_name.countDocuments()

Example
db.New.countDocuments()

```
DB_One> db.New.find()
[
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11e'),
    name: 'Arun',
    dept: 'HR',
    age: 30
  }
]
DB_One> db.New.countDocuments()
2
DB_One>
```

count() with conditions

Definition
Displays the count of records that satisfies a given condition.

Syntax
db.Collection_Name.find({condition}).count()

Example
db.New.find({dept:”CS”}).count()

```
DB_One> db.New.find({dept:"CS"}).count()
1
DB_One>
```

17. sort()

Definition
Sorts documents in ascending or descending order.

Syntax
db.collection_name.find().sort({
  field:1
})

Example (Ascending)
db.New.find().sort({
  age:1
})

```
DB_One> db.New.find().sort({age:1})
[
  {
    _id: ObjectId('6a1fc4baa3b58f3212abc119'),
    name: 'Vihaana',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11a'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11b'),
    name: 'Anvika',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11c'),
    name: 'Latchaya',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11e'),
    name: 'Arun',
    dept: 'HR',
    age: 30
  }
]
DB_One>
```

Example (Descending)

```
db.New.find().sort({
  age:-1
})
```

```
DB_One> db.New.find().sort({age:-1})
[
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11e'),
    name: 'Arun',
    dept: 'HR',
    age: 30
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  },
  {
    _id: ObjectId('6a1fc4baa3b58f3212abc119'),
    name: 'Vihaana',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11a'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11b'),
    name: 'Anvika',
    dept: 'IT',
    age: 20
  },
  {
    _id: ObjectId('6a1fc545a3b58f3212abc11c'),
    name: 'Latchaya',
    dept: 'IT',
    age: 20
  }
]
DB_One>
```

18. limit()

Definition

Restricts the number of documents returned.

Syntax

```
db.collection_name.find().limit(n)
```

Example

```
db.New.find().limit(1)
```

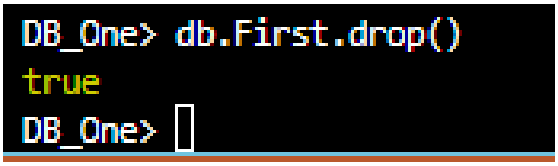
```
DB_One> db.New.find().sort({ name:1 })
[
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11e'),
    name: 'Arun',
    dept: 'HR',
    age: 30
  },
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  }
]
DB_One> db.New.find().limit(1)
[
  {
    _id: ObjectId('6a1fc944a3b58f3212abc11d'),
    name: 'Nivi',
    dept: 'CS',
    age: 25
  }
]
DB_One>
```

19. drop()

Definition
Deletes an entire collection.

Syntax
db.collection_name.drop()

Example
db.First.drop()



20. dropDatabase()

Definition
Deletes the current database.

Syntax
db.dropDatabase()

Example
db.dropDatabase()

```
DB_Two> db.dropDatabase()
{ ok: 1, dropped: 'DB_Two' }
DB_Two>
```

MongoDB Aggregation

Count

Definition
To determine the total number of documents in a collection or group ($sum: 1)

Count All Documents
Syntax

```
db.collection_name.aggregate([
  {
    $group: {
      _id: null,
      totalCount: { $sum: 1 }
    }
  }
])
```

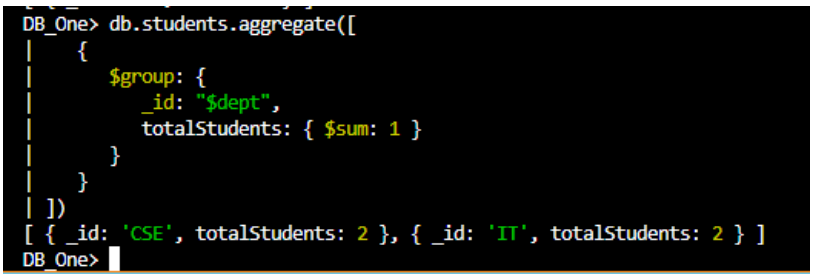
Example

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      totalCount: { $sum: 1 }
    }
  }
])
```

```
DB_One> db.students.aggregate([
|   {
|     $group: {
|       _id: null,
|       total: { $sum: 1 }
|     }
|   }
| ])
[ { _id: null, total: 4 } ]
DB_One>
```

count from each Department

```
db.students.aggregate([
  {
    $group: {
      _id: "$dept",
      totalStudents: { $sum: 1 }
    }
  }
])
```



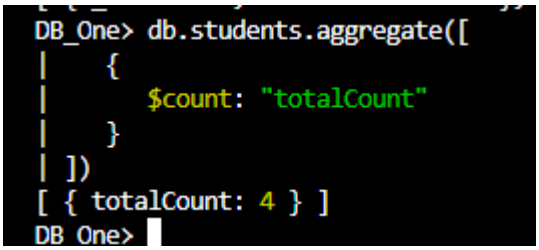
Alternative Count Method (MongoDB also provides $count)

Syntax

```
db.collection_name.aggregate([
  {
    $count: "totalCount"
  }
])
```

Example

```
db.students.aggregate([
  {
    $count: "totalCount"
  }
])
```



MAX

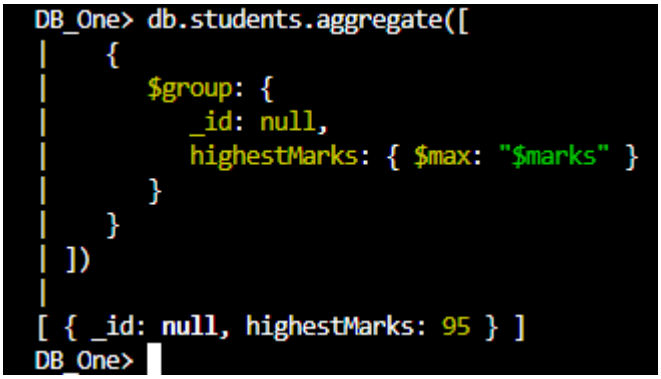
Definition
find the **largest value** in a field among all documents or within each group.

Maximum from All Documents
Syntax

```
db.collection_name.aggregate([
  {
    $group: {
      _id: null,
      maxValue: { $max: "$field_name" }
    }
  }
])
```

Example

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      highestMarks: { $max: "$marks" }
    }
  }
])
```



Maximum from each Department

```
db.students.aggregate([
 {
   $group: {
      _id: "$dept",
      maximum: { $max: "$marks" }
}
```

}])

```
DB_One> db.students.aggregate([ { $group: { _id: "$dept", maximum: { $max: "$marks" }}}])
[ { _id: 'CSE', maximum: 95 }, { _id: 'IT', maximum: 90 } ]
DB_One>
```

MIN

Definition
find the **smallest value** in a field among all documents or within each group.

Minimum from All Documents

Syntax

```
db.collection_name.aggregate([
  {
    $group: {
      _id: null,
      minValue: { $min: "$field_name" }
    }
  }
])
```

Example

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      lowestMarks: { $min: "$marks" }
    }
  }
])
```

Minimum from each Department

```
db.students.aggregate([
 {
   $group: {
      _id: "$dept",
      minimum: { $min: "$marks" }
}
}])
```

```
DB_One> db.students.aggregate([{ $group: { _id: "$dept" ,mini : { $min: "$marks"}}}])
[ { _id: 'CSE', mini: 80 }, { _id: 'IT', mini: 85 } ]
DB_One> db.students.aggregate([{ $group: { _id: null ,mini : { $min: "$marks"}}}])
[ { _id: null, mini: 80 } ]
DB_One>
```

SUM

Sum of All Documents

Syntax

```
db.collection_name.aggregate([
  {
    $group: {
      _id: null,
      sumValue: { $sum: "$field_name" }
    }
  }
])
```

Example

```
db.students.aggregate([
 {
    $group: {
        _id: null,
        sumValue: { $sum: "$marks"}
 }
} ] )
```

```
DB_One> db.students.aggregate([ { $group: { _id: null, sumValue: { $sum: "$marks"} } } ])

[ { _id: null, sumValue: 350 } ]
DB_One>
```

Sum from each department

Example

```
db.students.aggregate([
 {
    $group: {
        _id: “$dept”,
        sumValue: { $sum: "$marks"}
 }
```

}])

```
DB_One> db.students.aggregate([ { $group: { _id: "$dept", sum: { $sum : "$marks" }}}])
[ { _id: 'CSE', sum: 175 }, { _id: 'IT', sum: 175 } ]
```

AVG
AVG of all documents

Syntax

```
db.collection_name.aggregate([
  {
    $group: {
      _id: null,
      avgValue: { $avg: "$field_name" }
    }
  }
])
```

Example

```
db.students.aggregate([
 {
    $group: {
        _id: null,
        avgValue: { $avg: "$marks"}
 }
} ] )
```

```
[ { _id: null, sumValue: 350 } ]
DB_One> db.students.aggregate([
|  {
|      $group: {
|           _id: null,
|           avgValue: { $avg: "$marks"}
|  }
| } ] )
|
[ { _id: null, avgValue: 87.5 } ]
DB_One>
```

Avg from each department

Example

```
db.students.aggregate([
 {
    $group: {
        _id: “$dept”,
        avgValue: { $avg: "$marks"}
 }
} ] )
```

```
DB_One> db.students.aggregate([
|  {
|      $group: {
|           _id:"$dept"  ,
|           avgValue: { $avg: "$marks"}
|  }
| } ] )
|
[ { _id: 'CSE', avgValue: 87.5 }, { _id: 'IT', avgValue: 87.5 } ]
DB_One>
```

match

Definition
filter documents based on specified conditions.

Syntax

```
db.collection_name.aggregate([
  {
    $match: {
      condition
    }
  }
])
```

Example 1: Match Department = IT

```
db.students.aggregate([
  {
    $match: {
      dept: "IT"
    }
```

```
  }
])
```

```
DB_One> db.students.aggregate([ { $match: { dept:"IT"}}])
[
  {
    _id: ObjectId('6a1fe4cf84347b5679abc114'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20,
    marks: 90
  },
  {
    _id: ObjectId('6a1fe4cf84347b5679abc115'),
    name: 'Anvika',
    dept: 'IT',
    age: 21,
    marks: 85
  }
]
DB_One>
```

Example 2: Match Age > 20

```
db.students.aggregate([
  {
    $match: {
      marks: { $gt: 89 }
    }
  }
])
```

```
DB_One> db.students.aggregate([ { $match: { marks: { $gt: 89}}}])
[
  {
    _id: ObjectId('6a1fe4cf84347b5679abc114'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20,
    marks: 90
  },
  {
    _id: ObjectId('6a1fe4cf84347b5679abc116'),
    name: 'Arun',
    dept: 'CSE',
    age: 22,
    marks: 95
  }
]
DB_One>
```

Example 3: Match Using AND

```
db.students.aggregate([
  {
    $match: {
      dept: "IT",
      age: 20
    }
  }
])
```

```
DB_One> db.students.aggregate([ { $match: { dept:"IT",age:20}}])
[
  {
    _id: ObjectId('6a1fe4cf84347b5679abc114'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20,
    marks: 90
  }
]
DB_One>
```

Example 4: Match Using OR

```
db.students.aggregate([
  {
    $match: {
      $or: [
        { dept: "IT" },
        { age: 21 }
      ]
    }
  }
])
```

```
[ { _id: 'CSE', avgValue: 87.5 }, { _id: 'IT', avgValue: 87.5 } ]
DB_One> db.students.aggregate([
...     {
...         $match: {
...             $or: [
...                 { dept: "IT" },
...                 { age: 21 }
...             ]
...         }
...     }
... ])
...
[
  {
    _id: ObjectId('6a1fe4cf84347b5679abc114'),
    name: 'Yashvi',
    dept: 'IT',
    age: 20,
    marks: 90
  },
  {
    _id: ObjectId('6a1fe4cf84347b5679abc115'),
    name: 'Anvika',
    dept: 'IT',
    age: 21,
    marks: 85
  }
]
DB_One>
```